FINAL-YEAR PROJECT · ALIGNED WITH THE OWASP AI MODEL WATERMARKING PROJECT (NOT OFFICIAL OWASP WORK)

Sigil.

Sigil hides a secret mark inside a neural network's weights. If someone copies the model, the owner can prove it's theirs with a zero-knowledge proof that reveals nothing about the mark.

`64-bit mark in one layer` `accuracy 98.69% → 98.66%` `proof made in ~4 s` `checked in ~0.4 s`

Contents: The idea How it works The six demos Who does what Run it Limits

THE IDEA

Prove it's yours without giving away the secret

The problem

Trained models are valuable and easy to copy. If someone republishes your model as theirs, you need proof of ownership that doesn't hand them your secret.

The mark

While training, a small extra loss makes one layer's weights carry a 64-bit signature. The model works the same, and only the owner's secret key can read the bits.

The proof

The owner makes a zero-knowledge proof (Circom + Groth16). The verifier learns one fact: the registered owner's mark is in this exact file. Nothing else.

> **Our research finding**
>
> In earlier zero-knowledge ownership schemes the prover chooses the key, so a thief can fit a fake key to someone else's model and still get a valid proof (demos C and E). Sigil fixes this: the key and signature come from a secret seed that was committed and timestamped when the model was registered, and the circuit re-derives them itself. A key can't be invented afterwards (demos D and F).

HOW IT WORKS

Five steps, from training to a verdict

1. **Create an identity.**
   The owner makes an Ed25519 key pair and a secret random seed. Both stay encrypted on the owner's machine.
2. **Embed the mark.**
   The watermark key and the 64-bit signature are derived from the seed and the owner's key, then trained into one layer.
3. **Register.**
   The owner publishes a signed record: a commitment (hash) of the seed, the settings and the model file's SHA-256. A registry timestamps it, and the earliest registration wins a dispute.
4. **Open a claim.**
   Someone who suspects a copy opens a claim on that file, with a fresh random number (a nonce) so an old proof can't be reused.

5 **Prove and verify.**
The owner proves the committed mark is in the suspect file, with at most 8 of the 64 bits wrong. The verifier recomputes the file's fingerprint and checks the signature, the timestamp, the nonce and the proof. Every failed check gives its own reason.

THE SIX DEMOS

The story the app tells, A to F

DEMO	WHAT IT SHOWS	LIVE RUN
A **Commit & embed**	The owner publishes a hash of the secret seed and embeds the mark. Accuracy drops by about 0.03 points.	~5 s
B **Removal attacks**	Pruning, quantisation, fine-tuning, overwriting and permutation, and which of them remove the mark.	~10 s
C **Ownership forgery**	Without a commitment, a thief fits a fake key ("MALLORY!") to the owner's model.	~3 s
D **Committed seed**	The fix without zero knowledge: 0 of 20,000 forgery attempts succeed.	~3 s
E **Zero-knowledge proofs**	With EZKL, the thief's forged proof verifies under the owner's own key. This is why the commitment matters.	~10 s
F **Protocol v2**	The full fix: a real Circom + Groth16 proof, every verifier check, the thief, misuse attempts, attacks and the overwrite dispute.	~80 s

WHO DOES WHAT

Four owners, one system

Roles follow the project plan (REQUIREMENTS.md). Everyone should be able to explain any part; these are the areas each person answers for.

Sami Daire

PROJECT LEAD · ZK PROOFS · VERIFICATION

OWNS

Protocol v2 (seed commitment, owner identity, registration and timestamps), the Circom circuit and its Groth16 setup, the verifier's checks, the on-chain verifier, and putting the pieces together.

EXPLAINS

Demos C, D, E and F. Why a committed seed stops forgery, and what the proof does and doesn't show.

Mohamed Ansari

AI & WATERMARKING · ROBUSTNESS

OWNS

Embedding the mark in PyTorch, Keras and ONNX models, the attack suite, setting the acceptance threshold from 30 never-watermarked models, and the trigger-set mark.

EXPLAINS

Demos A and B and the Robustness lab. Which attacks the mark survives, which remove it, and what accuracy the thief gives up.

Arfat Shaikh

FRONTEND · SDK & CLI · DESIGN

OWNS

The web app (home page, dashboard, demo pages, the Model under test bar), packaging the `sigil` SDK and its command line, and the example notebooks.

EXPLAINS

The walk-through of the app, and how a developer would use the SDK on their own model.

Yash Bandiwadekar

BACKEND · FASTAPI · POSTGRESQL

OWNS

The demo server (one job at a time, cancel, timeouts, local-only security) and the FastAPI + PostgreSQL backend: accounts and roles, registry, verification history, a tamper-evident audit log, reports and deployment monitoring.

EXPLAINS

How the app talks to the server, what gets stored, and why the server only listens on this machine.

RUN IT

Operate the project, step by step

Run every command from the project folder (the one with DEPLOY.md). Everything runs locally; nothing is sent to a cloud service.

Option 1: replay mode

`no install · about 1 min · needs Python 3.10+`

Run buttons replay recorded real runs. Use this as the backup during a presentation.

```
# or double-click Start-Demo-UI-Simulate.cmd
python demo\server.py --simulate
```

Option 2: live mode

`5–15 min setup · needs Python 3.10+ and Node.js 20.19+`

1. **Install.**
   Creates `.venv` and installs PyTorch, the SDK, the circuit tools and the proving keys.

   ```
   .\setup.ps1 --demo              # macOS / Linux: ./setup.sh --demo
   ```

2. **Check.**
   Runs the tests, starts the app and makes a real proof.

   ```
   .venv\Scripts\python scripts\verify_deploy.py
   ```

   `It must end with: ALL CHECKS PASSED`

3. **Start.**
   Double-click `Start-Demo-UI.cmd`, or run:

   ```
   .venv\Scripts\python demo\server.py --open
   ```

4. **Open the app.**
   Go to `http://127.0.0.1:8765` and press **Open dashboard**. The sidebar shows the GPU, or "CPU only".

Presenting the demo

1. Press **Clear results** so every page starts empty. Everything that appears next comes from runs the audience watches.
2. Press **Run all live** (fills demos A–E, about 30 s), then **Run demo F live** (about 80 s; makes and verifies a real proof).

3. Follow the route: Overview → Embed watermark → Verify ownership (try a misuse) → Robustness lab → demos A–F. The ← → keys move between demos.
4. In the **Model under test** bar, pick a model and press **Test this model**. Good contrasts: *pruned 90%* (ownership still proven), *channels permuted* (mark gone), *clean.pt* (never marked).
5. Press **Clear results** again before the next group.

Don't run live: Retrain (it replaces the owner key) and E3 (it needs about 32 GB of RAM).

Backup: keep `python demo\server.py --simulate --port 8766` ready in a second terminal.

For developers: the command line

```
# owner
sigil keygen                                    # owner key, encrypted
sigil seed --bits 64 --layer conv2              # secret seed + settings
sigil embed model.pt --arch mymodels:Net --data train.npz
sigil register model.wm.pt --registry-key secrets/registry.key
# verifier opens a claim, owner proves, verifier checks and gets a report
sigil claim registration.json --suspect suspect.pt
sigil prove suspect.pt --claim claim.json --out bundle
sigil report bundle --suspect suspect.pt --registration registration.json --receipt
registration.receipt.json --trusted-registry <registry key>
```

IF YOU SEE	DO THIS
"Can't reach the demo server"	Start the server. If port 8765 is taken, add `--port 8770`.
"No ML stack: replay only"	Start with the `.venv` Python (Option 2), or present in replay mode.
Demo F says the circuit isn't set up	Re-run `setup.ps1 --demo`; it installs the proving keys.
Antivirus blocks `circom.exe`	It's the official circom 2.2.3 release. Allow it, or show demo F in replay mode.

LIMITS

What we tell examiners up front

- **Model extraction removes the mark.** Training a fresh copy from the model's answers loses the weight mark, and in our tests the trigger-set mark too.
- **Permutation hides it.** Reordering a layer's channels hides the mark unless the owner realigns the model first.
- **64 bits is the demo size.** A thief's seed search costs about 2^{32} tries at 64 bits, which a GPU can do. The secure 256-bit setting raises it to about 2^{120} (proof in about 12 s). Timestamps still decide disputes.
- **Groth16 needs a trusted setup.** We use a public Powers of Tau file plus our own contribution.

More detail in the repository: `DEMO.md` (presenter runbook), `Sigil-User-Guide.pdf` (illustrated guide), `docs/protocol.md` (the protocol) and `OWASP AI Model Watermarking.pptx` (slides).